1. What is Azure Application Insights?

Azure Application Insights is an **Application Performance Management (APM)** service within **Azure Monitor**. It enables you to **monitor, analyze, and diagnose** the behavior and performance of live applications—particularly **ASP.NET / ASP.NET Core, APIs, microservices, and Azure-hosted workloads**.

It answers operational questions such as:

- Is my application healthy?
- Are users experiencing failures or slowness?
- Which APIs or dependencies are failing?
- Where exactly is the exception occurring in code?

In practical terms, Application Insights collects **telemetry** from your application and stores it in Azure for **querying, alerting, and visualization**.

Note: **Telemetry** means **automatically collecting, transmitting, and analyzing data about the behavior and health of a system from a remote location**.

---

2. What Telemetry Does Application Insights Collect?

By default, Application Insights captures the following **without code changes**:

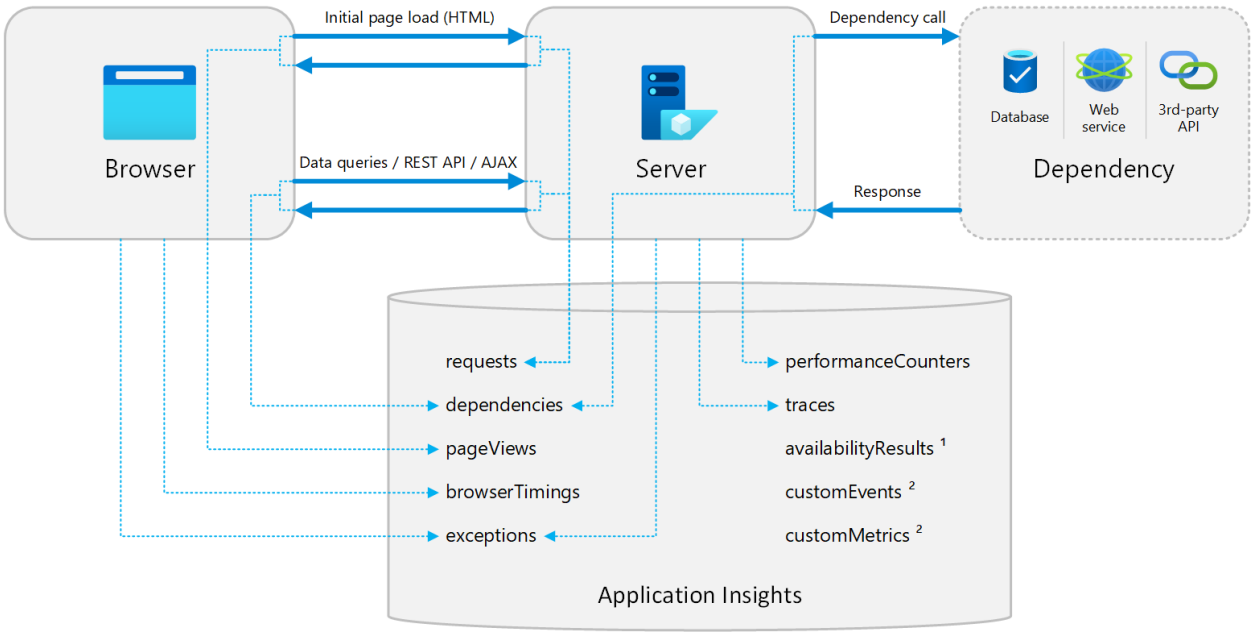
1. **Request Telemetry**
    - HTTP request count
    - Response time
    - HTTP status codes (200, 500, etc.)
2. **Dependency Telemetry**
    - SQL Server
    - Azure SQL
    - HTTP calls (REST, external APIs)
    - Service Bus, Storage, etc.
3. **Exception Telemetry**

- Unhandled exceptions
- Stack traces
- Inner exceptions

4. **Performance Counters**
   - CPU usage
   - Memory consumption
   - Request throughput
5. **Availability & Health**
   - Availability tests
   - Failure rates
6. **Custom Telemetry**
   - Custom events
   - Custom metrics
   - Custom traces

---

3. How Application Insights Works (High-Level Flow)



1. Your application runs with the **Application Insights SDK**
2. Telemetry is **automatically collected**
3. Data is sent to **Azure Monitor ingestion endpoint**
4. Telemetry is stored in a **Log Analytics workspace**
5. You analyze data using:
    - Azure Portal dashboards
    - **Kusto Query Language (KQL)**
    - Alerts and workbooks

---

4. When Should You Use Application Insights?

Use Application Insights when you need:

- Production monitoring
- Root cause analysis
- Performance optimization
- Centralized logging for APIs and microservices
- Correlation across services (Distributed Tracing)

This aligns well with **Clean Architecture** and **Microservices** setups you have been working with.

---

5. How to Configure Application Insights (ASP.NET Core API)

Step 1: Create Application Insights Resource

1. Open **Azure Portal**
2. Search **Application Insights**
3. Click **Create**
4. Fill:
    - Resource Group

- Name
- Region
- Workspace-based (recommended)

5. Click **Review + Create**

After creation, note the **Connection String** (preferred over Instrumentation Key).

Step 2: Add NuGet Package

In your ASP.NET Core API:

```
dotnet add package Microsoft.ApplicationInsights.AspNetCore
```

Step 3: Configure in Program.cs (.NET 6+)

```
builder.Services.AddApplicationInsightsTelemetry(options =>
{
    options.ConnectionString =
        builder.Configuration["ApplicationInsights:ConnectionString"];
});
```

Step 4: Add Configuration in appsettings.json

```
{
  "ApplicationInsights": {
    "ConnectionString": "InstrumentationKey=xxxx;IngestionEndpoint=https://xxxx"
  }
}
```

Best practice: store this in **Azure Key Vault** (similar to how you store DB connection strings).

Step 5: Verify Automatic Telemetry

Run your API and make some requests.

In Azure Portal → Application Insights:

- **Live Metrics**
- **Failures**
- **Performance**
- **Application Map**

You should see:

- Incoming HTTP requests
- Response times
- Dependency calls (SQL, HTTP)

---

6. Logging Integration with ILogger (Recommended)

Application Insights integrates seamlessly with ILogger.

```
private readonly ILogger<ProductController> _logger;


public ProductController(ILogger<ProductController> logger)
{
    _logger = logger;
}


_logger.LogInformation("Fetching product with id {ProductId}", id);
_logger.LogError(ex, "Error while fetching product");
```

These logs appear in **Logs (Analytics)**: